

Primera práctica SAPP

Vulnerabilidades en *Amazoncillo*

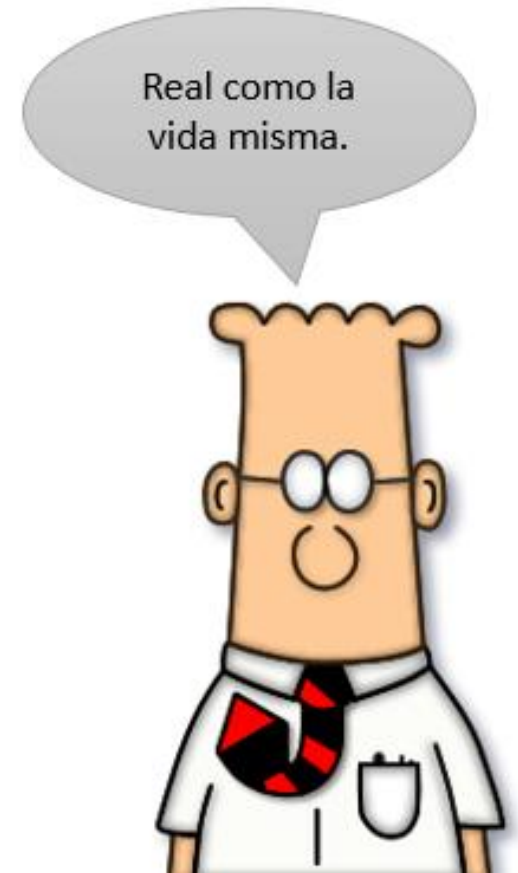


Vistazo global

- **Objetivo:** analizar una aplicación web para encontrar y arreglar (muchas de) las vulnerabilidades de seguridad que se estudian en teoría.
 - Deficiencias que se pueden aprovechar para causar algún tipo de daño, engaño, robo de información, denegación de servicio, etc.
- **Punto de partida:** se proporciona una aplicación web desarrollada en Java con arquitectura Modelo-Vista-Controlador.
- **Entrega:**
 - Una nueva versión de la aplicación con las vulnerabilidades arregladas.
 - Un documento en el que se detallan las vulnerabilidades encontradas, cómo se arreglarían y 3 *exploits* de calidad.

Intrahistoria

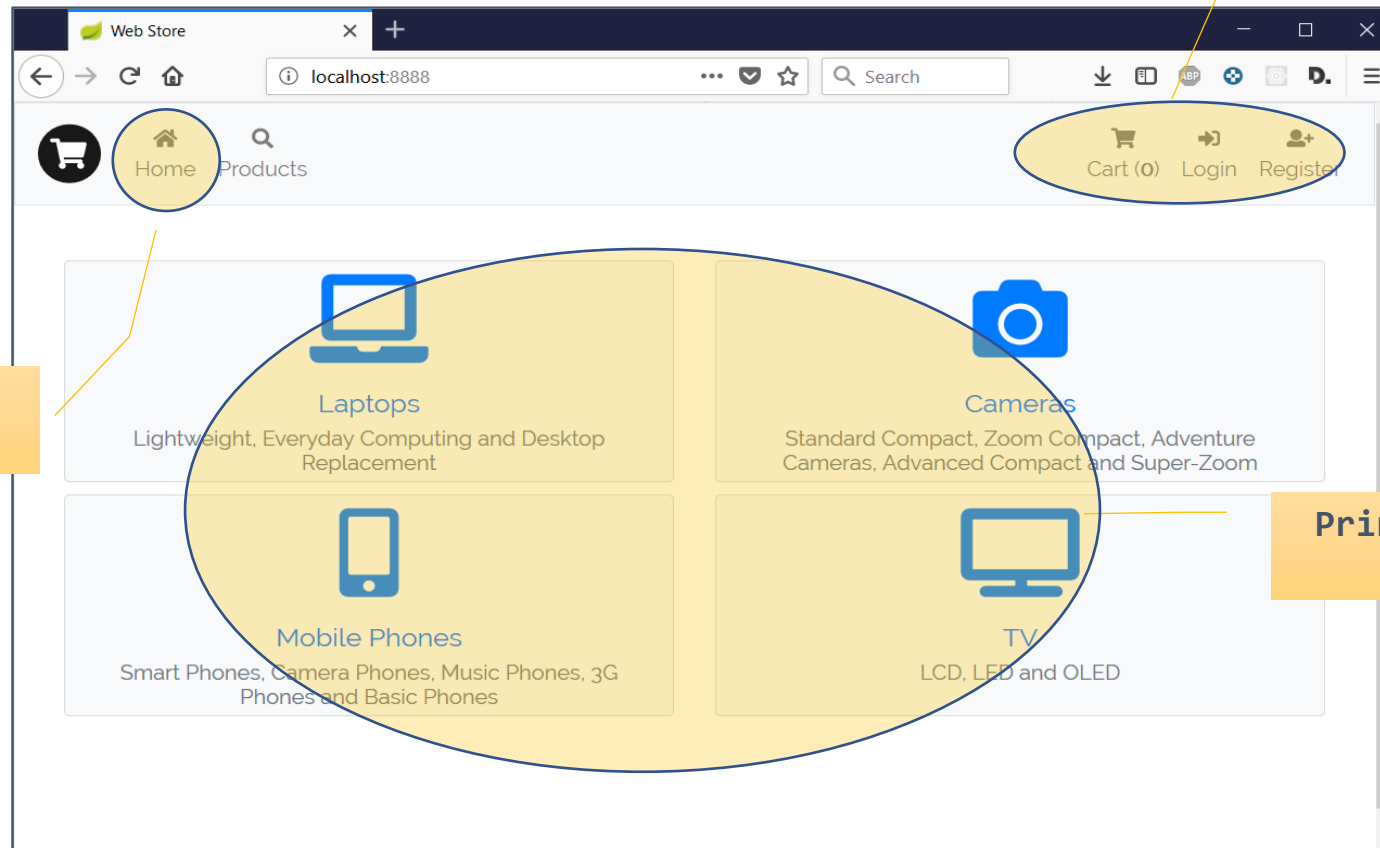
- La empresa *Amazoncillo* ha decidido abrir una nueva sucursal de su tienda de comercio electrónico, contratando el desarrollo de la aplicación web a *BonitoYBarato, Inc.*
- Una vez terminada, justo antes de poner la tienda en línea, desde la empresa matriz solicitan un informe de seguridad.
- En *Amazoncillo* habían omitido cualquier preocupación específica por la seguridad, y por ello contratan a la empresa especializada *Fortifícame*, líder mundial en la materia.
- Terminado el análisis preliminar, *Fortifícame* informa de que la cantidad de vulnerabilidades es tan alta que el coste de arreglarlas supera ampliamente el coste del desarrollo inicial.



Una tienda *online* tradicional

- Permite a los usuarios darse de alta y modificar su perfil.
- Permite recordar los datos del usuario en el navegador local y la posibilidad de reestablecer la contraseña cuando el usuario se ha olvidado de ella.
- Los usuarios no registrados sólo pueden ver los productos sin posibilidad de comprar.
- Por simplicidad, la cesta de la compra se almacena en la sesión del usuario (así pues, los datos se pierden si se reinicia el servidor).
- El usuario puede realizar un pedido a partir de los productos de la cesta de la compra.
- El proceso de pago no está implementado y asumimos que la transacción bancaria se realiza de forma correcta una vez se formaliza el pedido.
- A la hora de pagar un pedido, es posible almacenar la tarjeta de crédito para futuras compras.
- No es necesario pagar en el momento de hacer la compra y se pueden cancelar los pedidos que todavía no han sido pagados.
- Los usuarios pueden valorar los productos que han comprado y pueden ver todas las valoraciones de un producto antes de comprarlo.

Aplicación (1): página principal

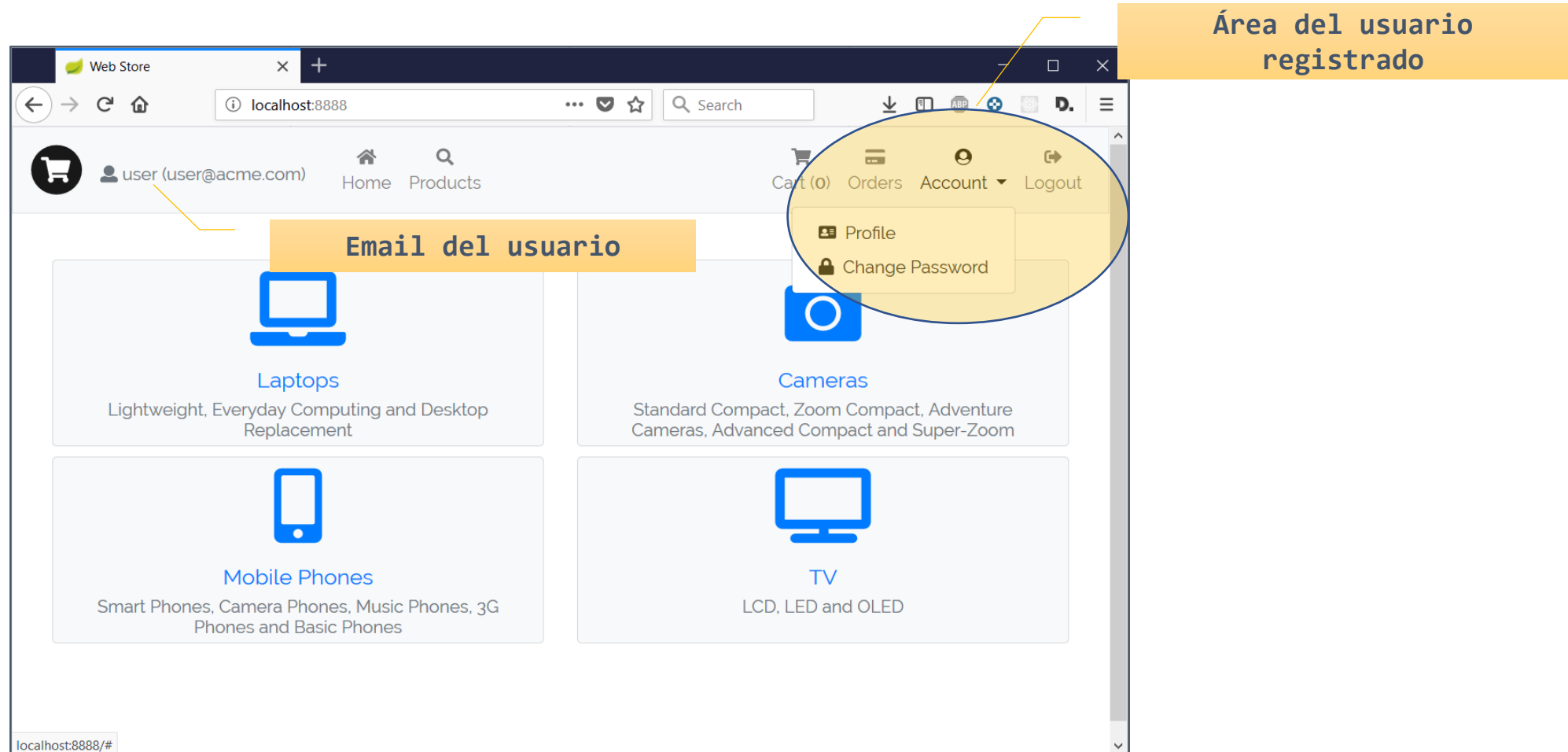


Área del usuario
sin registrar

Listado de todos los
productos

Principales categorías
de productos

Aplicación (2): página principal usuario registrado



Aplicación (3): registro de un nuevo usuario

Web Store

localhost:8888/registration

Home Products Cart (0) Login Register

New User Registration

Name

Email address We'll never share your email with anyone else

Password

Confirm Password

Address

Profile image No file selected. This image will be publicly shown in your product reviews

Enlace al formulario

Formulario de registro

Aplicación (4): modificar perfil de usuario

Web Store

localhost:8888/profile

user (user@acme.com) Home Products Cart (0) Orders Account Logout

Update User Profile

Name

Email address We'll never share your email with anyone else

Address

Profile image No file selected.

Current image

Descargar imagen de perfil

Eliminar imagen de perfil

This image will be publicly shown in your product reviews

Aplicación (5): autenticación

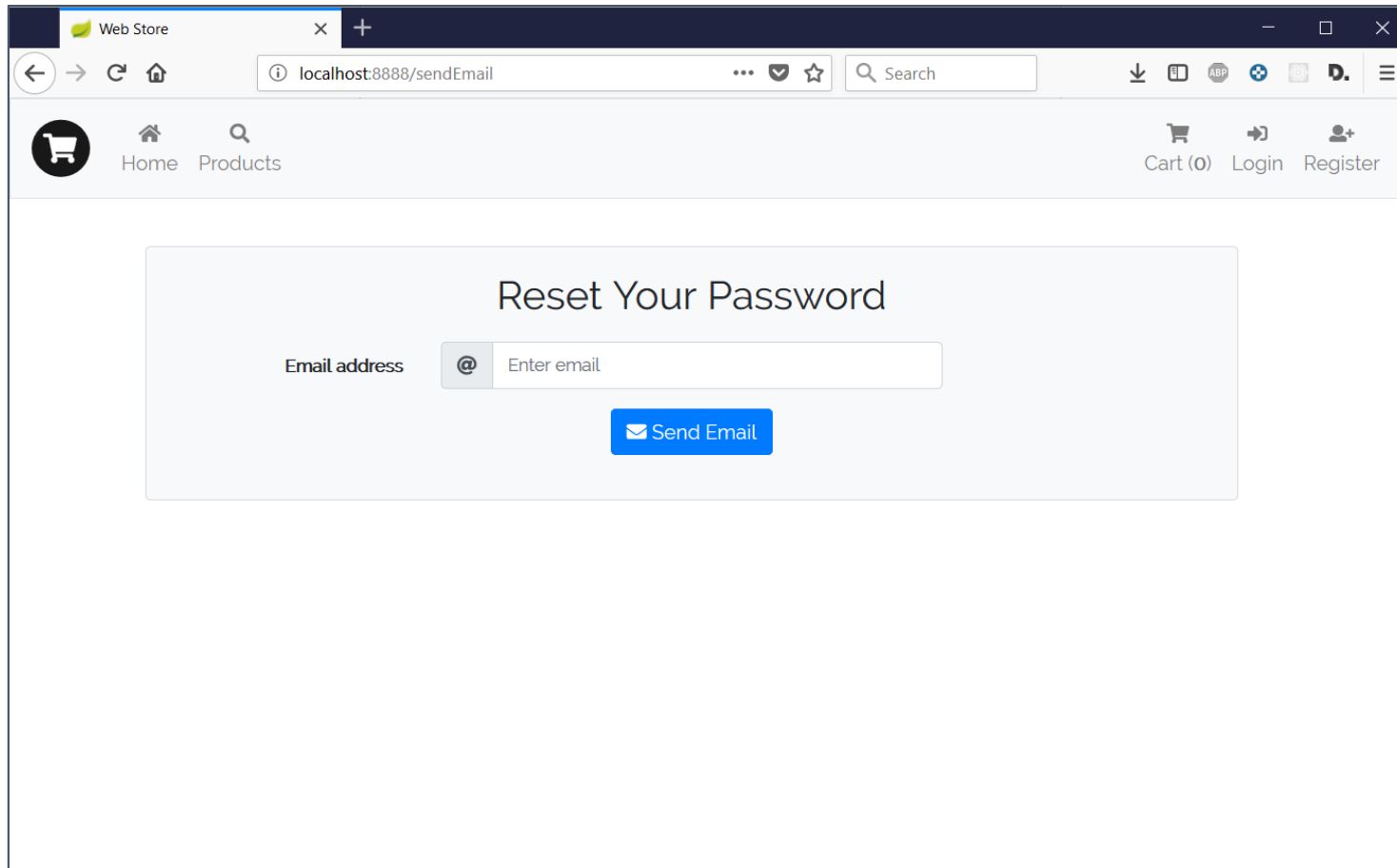
The screenshot shows a web browser window titled 'Web Store' with the address bar displaying 'localhost:8888/login'. The page features a navigation bar with 'Home' and 'Products' links, and a right sidebar with 'Cart (0)', 'Login', and 'Register' links. The 'Login' link is circled in blue. The main content area is a 'Login' form with the following elements:

- Email address:** A text input field with a placeholder '@ Enter email'.
- Password:** A text input field with a placeholder 'Enter your password'.
- ☐ Remember me in this computer
- [Forgot your password?](#)
-

Annotations with yellow boxes and lines pointing to specific features:

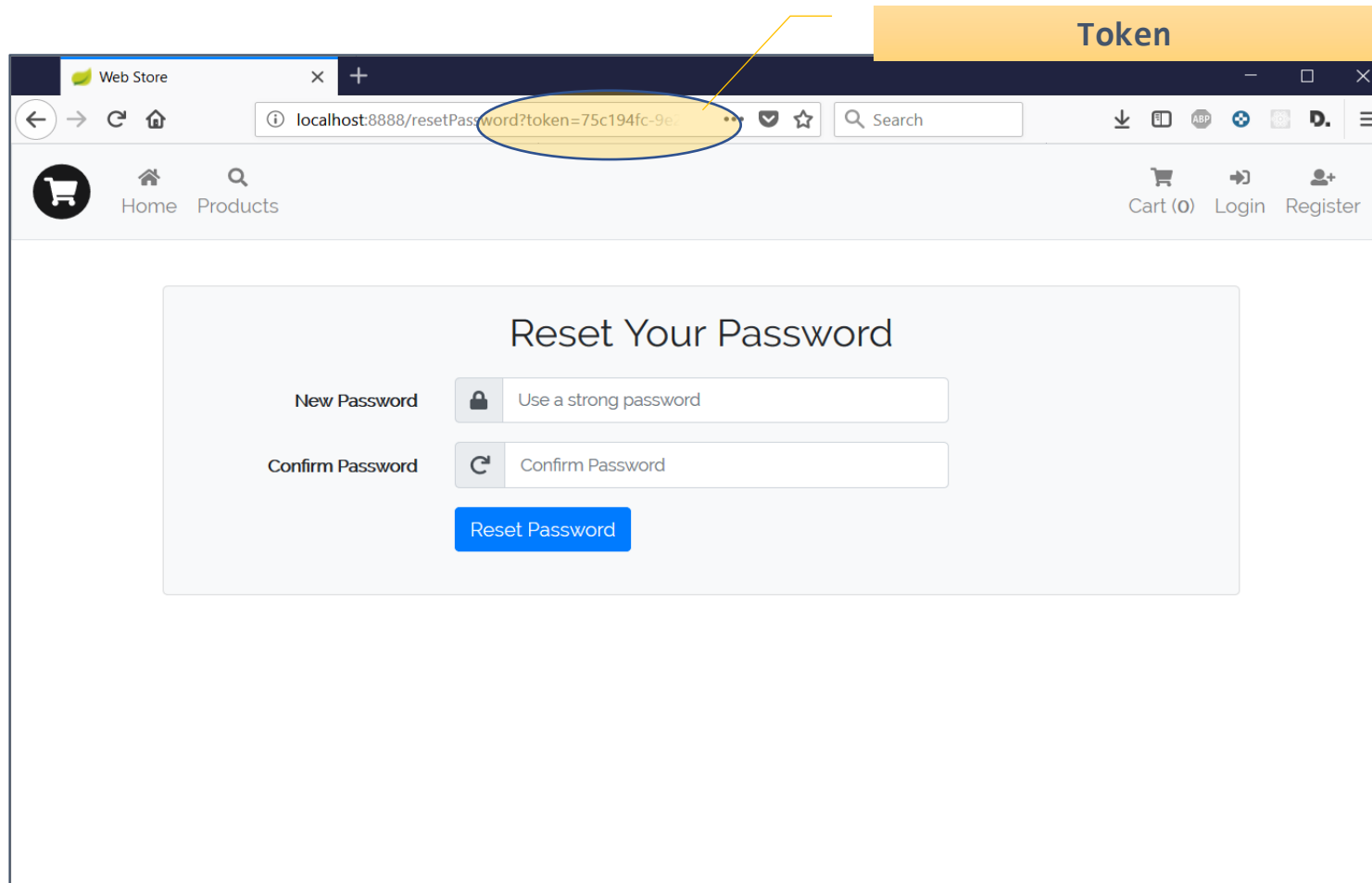
- Enlace al formulario:** Points to the 'Login' link in the top right navigation bar.
- Cookie persistente:** Points to the 'Remember me in this computer' checkbox.
- Correo electrónico para reestablecer contraseña:** Points to the 'Forgot your password?' link.

Aplicación (6): resetear contraseña



The screenshot shows a web browser window with the title 'Web Store'. The address bar displays 'localhost:8888/sendEmail'. The browser's navigation bar includes icons for back, forward, refresh, and home, along with a search bar. The website's header features a shopping cart icon, 'Home', 'Products', 'Cart (0)', 'Login', and 'Register' links. The main content area displays a 'Reset Your Password' form. The form has a title 'Reset Your Password' and a label 'Email address' next to an input field with a placeholder '@ Enter email'. Below the input field is a blue button labeled 'Send Email' with an envelope icon.

Aplicación (7): resetear contraseña



Aplicación (8): cambiar contraseña

Web Store

localhost:8888/changePassword

user (user@acme.com) Home Products Cart (0) Orders Account Logout

Profile
Change Password

Change Password

Old Password

New Password

Confirm Password

Login

localhost:8888/changePassword

Enlace al formulario

Aplicación (9): listado de productos

Web Store

localhost:8888/products/?category=Laptops

Home Products

Categoría

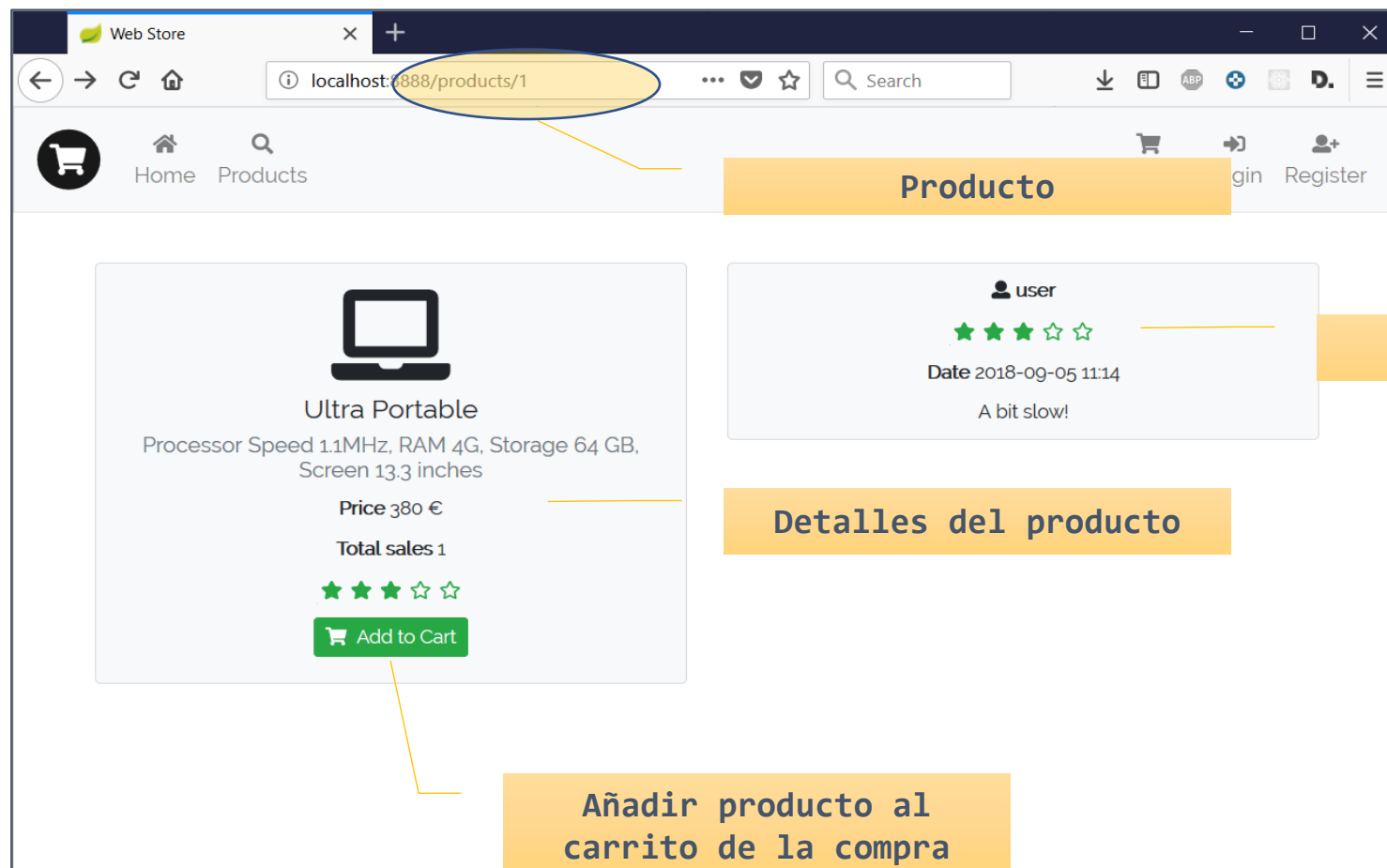
Show 10 entries

Search:

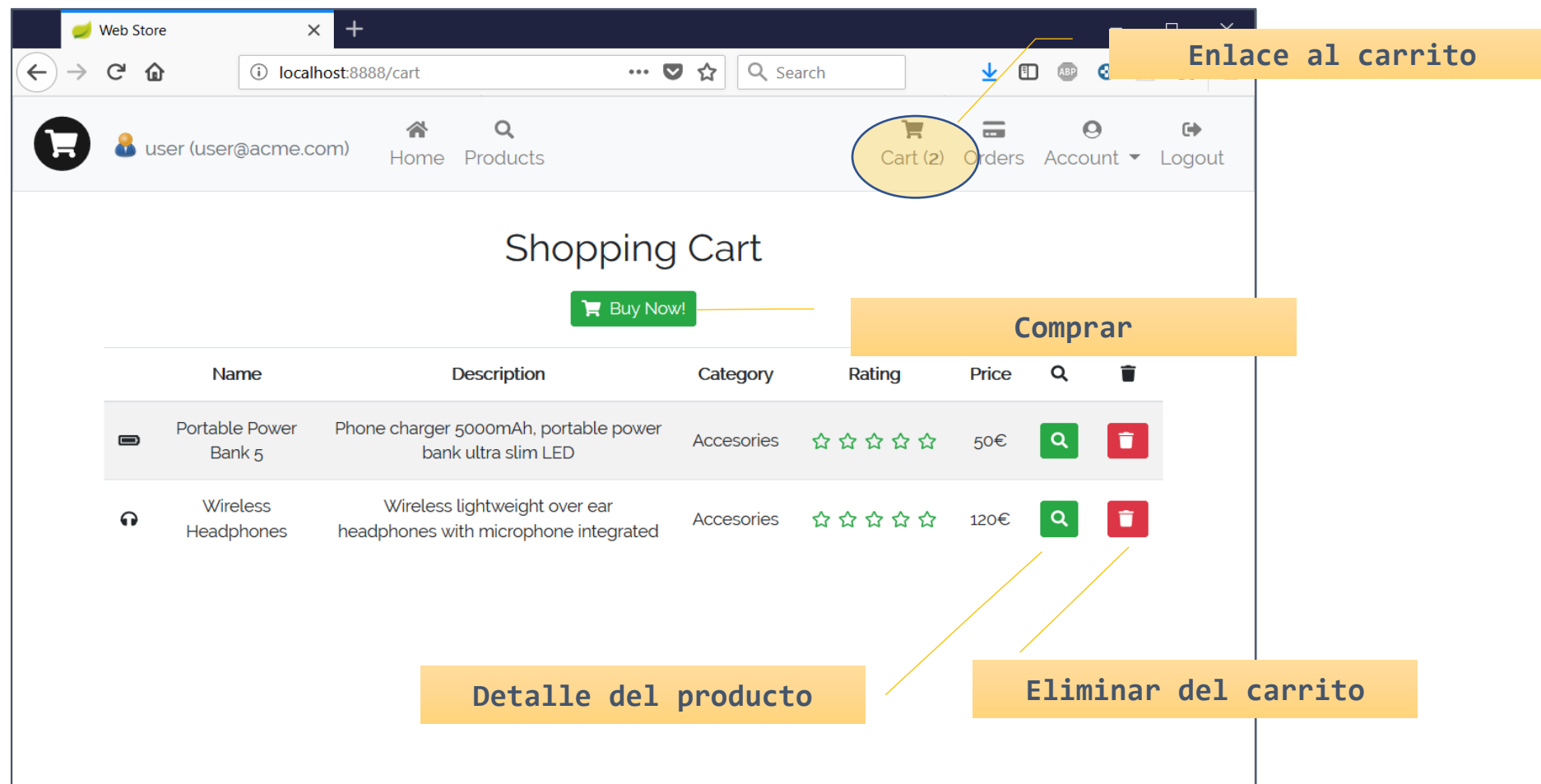
Name	Description	Category	Rating	Total sales	Price
Ultra Portable	Processor Speed 1.1MHz, RAM 4G, Storage 64 GB, Screen 13.3 inches	Laptops	☆☆☆☆☆	0	380€
General Purpose	Processor Speed 2.4MHz, RAM 4G, Storage 1 TB, Screen 15.6 inches	Laptops	☆☆☆☆☆	0	420€
Business Small	Processor Speed 2.4MHz, RAM 4G, Storage 128 GB, Screen 13.3 inches	Laptops	☆☆☆☆☆	0	535€
Business	Processor Speed 2.4MHz, RAM 8G, Storage 512 GB, Screen 14 inches	Laptops	☆☆☆☆☆	0	780€
Gaming	Processor Speed 2.8MHz, RAM 16G, Storage 2 TB, Screen 15.6 inches	Laptops	☆☆☆☆☆	0	925€
Ultra Power	Processor Speed 2.4MHz, RAM 16G,	Laptops	☆☆☆☆☆	0	1000€

Detalle del producto

Aplicación (10): detalle de un producto



Aplicación (11): carrito de la compra



Aplicación (12): comprar

Web Store

localhost:8888/orders/complet 90% Search

user (user@acme.com) Home Products Cart (2) Orders Account Logout

Complete Your Order

Price 170€

☒ Send to your default address

☐ Send to another address

Address

Name	Description	Category	Rating	Price
 Portable Power Bank 5	Phone charger 5000mAh, portable power bank ultra slim LED	Accesories	☆☆☆☆☆	50€
 Wireless Headphones	Wireless lightweight over ear headphones with microphone integrated	Accesories	☆☆☆☆☆	120€

Dirección de envío

Opciones de pago

Aplicación (13): listado de pedidos

Web Store

localhost:8888/orders

90%

Search

user (user@acme.com)

Home Products

Cart (0) Orders Account Logout

Your Orders

Order	Date	Price	State	
Portable Power Bank 5 (Accessories) and 1 more products	2018-09-05 11:58	170€		
Ultra Portable (Laptops)	2018-09-05 11:14	380€	The payment is pending	

Enlace al listado

Estado del pedido

Detalles del pedido

Aplicación (14): detalles del pedido

Web Store

localhost:8888/orders/2

user (user@acme.com) Home Products **Pedido** Logout

Your Order Portable Power Bank 5 (Accesories) and 1 more products

Price 170€

Date 2018-09-05 11:58

Address Campus de Elviña s/n

Pagar **Pay Now!** **Cancel Order!** **Cancelar pedido**

	Name	Description	Category	Rating	Price	
	Portable Power Bank 5	Phone charger 5000mAh, portable power bank ultra slim LED	Accesories	☆☆☆☆☆	50€	
	Wireless Headphones	Wireless lightweight over ear headphones with microphone integrated	Accesories	☆☆☆☆☆	120€	

Aplicación (15): pagar pedido

Web Store

localhost:8888/orders/3/pay

user (user@acme.com) Home Products Cart (0) Orders Account Logout

Complete Your Order

☐ Pay with your default credit card #1111

☒ Pay with another credit card

CVV

Expiration Month

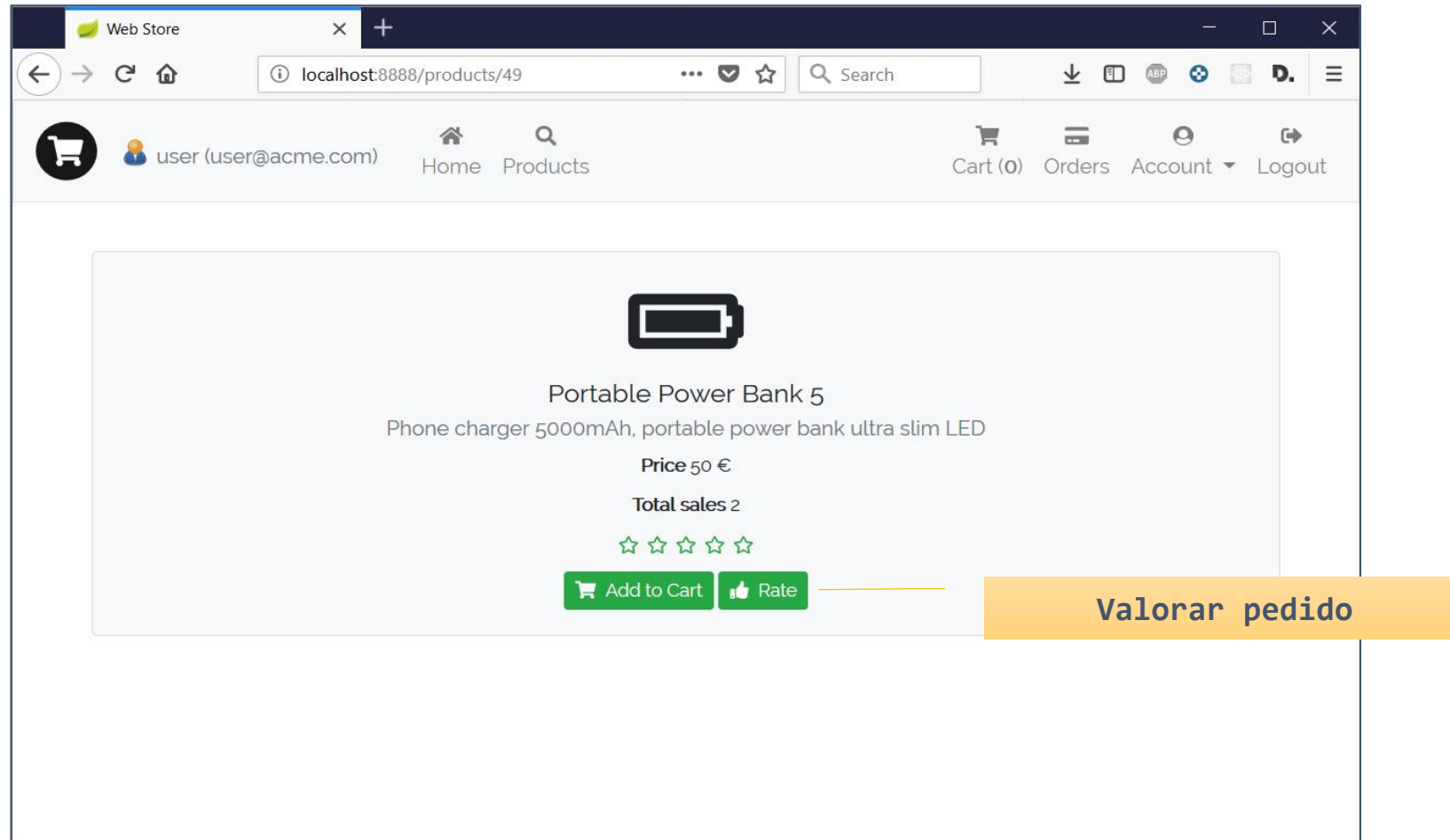
Expiration Year

☐ Save as your default credit card

Tarjeta de crédito guardada

Guardar tarjeta de crédito

Aplicación (16): detalle del producto comprado



Aplicación (17): valorar producto

Web Store

localhost:8888/products/49/rate

Search

user (user@acme.com) Home Products Cart (0) Orders Account Logout

New Review: Portable Power Bank 5 (Accesories)

Rate this product

★★★★☆

Review Details

Add Review

Puntuación 0-5

Comentario de texto

Detalles de implementación y distribución

- La aplicación ha sido implementada en Java siguiendo la arquitectura Modelo-Vista-Controlador y puede contener vulnerabilidades en cualquiera de esos tres subsistemas:
 - El **modelo** (lógica de negocio) ha sido implementado utilizando Hibernate + JPA (Java Persistence API). Los datos se almacenan en una base de datos Derby (local) que ya está preconfigurada.
 - La **vista** utiliza plantillas desarrolladas con Thymeleaf además de JQuery y Bootstrap como librerías web.
 - El **controlador** ha sido implementado utilizando la librería Spring MVC.
- La aplicación se distribuye como un proyecto Maven y utiliza Spring Boot para simplificar la configuración y facilitar su ejecución.

Sobre la configuración

- Se debe considerar la aplicación como terminada y lista para su puesta en funcionamiento.
- En consecuencia, la configuración también debería estar preparada para entornos en producción.
- Excepciones que no se considerarán vulnerabilidades:
 - Para facilitar el desarrollo de la práctica, el proyecto se recarga automáticamente tras la modificación de cualquier fichero.
 - Algunas categorías de log también se encuentran activadas de forma intencionada para facilitar la visualización de las excepciones.
 - La aplicación no implementa mecanismos de control contra ataques por fuerza bruta, por lo tanto, tampoco se considerarán este tipo de vulnerabilidades.

Entrega

- **Vulnerabilidades corregidas:**
 - Incluir el proyecto Maven, pero sin ficheros binarios ni compilados.
- **Documentación:**
 - Descripción y localización de cada vulnerabilidad.
 - Solución, con indicación de los principios teóricos aplicados.
 - 3 *exploits* de calidad: secuencia de pasos para causar el mayor daño posible.
 - Se probarán única y exclusivamente los pasos que se encuentren en la documentación.
- **Plazo:** miércoles, 11 de noviembre.
- **Defensa:** jueves 12 de noviembre (UDC), viernes 13 de noviembre (UVIGO) en horario de clase.

Ejemplo de plantilla para documentación

Vulnerabilidad	Inyección de código SQL
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Consecuencias	Cualquier usuario se puede autenticar sin necesidad de conocer la contraseña. Para explotar la vulnerabilidad tan sólo es necesario conocer el nombre del usuario.
Localización	Login.java:326, UserDAO.java:27 localizados en el paquete Java “es.mucs.store.business.user”
Solución	Como no es viable utilizar técnicas de “lista blanca”, se ha utilizado la técnica de escapado de las entradas de usuario. Concretamente, se han escapado los siguientes caracteres especiales en SQL: el punto y coma, las comillas dobles y las comillas simples.
Otra información	La vulnerabilidad ha sido encontrada de forma automática por la herramienta OWASP ZAP.

Evaluación

- Cada vulnerabilidad encontrada, solucionada y documentada correctamente se valorará con hasta 0,5 puntos, hasta un máximo de 7 puntos sobre 10.
 - No hay límite en el número de vulnerabilidades que se pueden presentar.
 - Reportar vulnerabilidades que no son tales (por ejemplo, falsos positivos de herramientas SAST y DAST) repercutirá restando 0,5 puntos.
- Para cada vulnerabilidad se valorarán por igual:
 - Localización y descripción (0,25).
 - Solución (0,25): para obtener la máxima puntuación, es necesario encontrar y arreglar todas las ocurrencias de la vulnerabilidad.
- 3 *exploits*, cada uno de ellos hasta 1 punto. Se valorará el daño causado y la cantidad de vulnerabilidades utilizadas. Un mismo tipo de vulnerabilidad sólo se podrá usar en 2 *exploits* a excepción de sql-i que sólo se podrá usar una vez.



- Se premiará con puntuación adicional al grupo que localice cada una de las vulnerabilidades en primer lugar.
 - Si ocurre en varios sitios, se premiará la primera ocurrencia.
 - Las vulnerabilidades que afectan al control de acceso se consideran problemas diferentes, por lo que se puede obtener un bonus por cada una de ellas.
 - Una vez localizado el problema, el grupo debe ponerlo en conocimiento del profesor de prácticas a través de un correo electrónico, aunque en ese momento se encuentren en la clase de prácticas y se lo pueda comunicar de forma presencial.
- También se premiará al primer grupo que reporte nuevas entradas en el CVE que afecten a alguna de las librerías que se usan en la aplicación web.
 - Tienen que ser entradas dadas de alta durante el período de realización de la práctica.
- **¡Cuidado!** Habrá penalizaciones por notificación de vulnerabilidades que no son tales o de entradas del CVE que no siguen los criterios establecidos.

Organización del laboratorio

- La práctica se realizará de forma preferente en grupos de 3 personas.
- Los grupos deben estar formados antes de que finalice el mes de septiembre.
- En el laboratorio trataremos de dedicar 1/3 de cada sesión a nuevas explicaciones y a fijar conceptos:
 - Introducción a herramientas de apoyo como SonarQube o ZASP.
 - Repaso de vulnerabilidades detectadas por la mayoría de los grupos, ...

Software requerido

- AdoptOpenJDK 21:
 - <https://adoptium.net/>
- Maven:
 - <https://maven.apache.org/download.cgi>
- Entorno de desarrollo:
 - Eclipse: <https://www.eclipse.org/downloads/packages/>
 - Netbeans: <https://netbeans.apache.org/download/index.html>
 - IntelliJ Idea: <https://www.jetbrains.com/es-es/idea/download>

Instalación simplificada

- Sólo es necesario realizar los siguientes pasos:
 - Instalar Java.
 - Instalar el entorno gráfico.
 - Importar el proyecto desde el entorno gráfico.
- Pequeño inconveniente: será necesario realizar todas las ejecuciones de maven desde el entorno gráfico.

Instalación de Java en Windows

- Se recomienda utilizar la última versión disponible de Java 21.
- Descargar la distribución y ejecutar el instalador.
- Establecer variable de entorno **JAVA_HOME** en *Panel de Control -> Sistema y Seguridad -> Editar las variables de entorno del sistema*.

JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-21.x.x-hotspot

- Añadir el directorio **bin** de la distribución de Java a la variable de entorno **PATH**.
- Se recomienda añadirlo en la primera posición para evitar problemas con otras instalaciones de Java que puedan existir en el sistema.

PATH = %JAVA_HOME%\bin;...

Instalación de Maven en Windows

- Descargar y descomprimir el fichero **ZIP** de la siguiente dirección

`https://maven.apache.org/download.cgi`

- Establecer la variable de entorno **MAVEN_HOME**

`MAVEN_HOME = C:\Program Files\Java\apache-maven-3.x.x`

- Añadir el directorio **bin** de la distribución de Maven a la variable de entorno **PATH**:

`PATH = %JAVA_HOME%\bin;%MAVEN_HOME%\bin; ...`

Instalación de Java en Linux/Mac

- Se recomienda utilizar la última versión disponible de Java 21.
- Se recomienda descomprimir la distribución en el directorio **/opt** (en Linux).
- Establecer la variable de entorno **JAVA_HOME** en el fichero **.bashrc** (en Mac OS X utilizar el fichero **.zshrc** o **.bash_profile**) localizado en la **\$HOME** del usuario.

```
export JAVA_HOME=/opt/jdk-21.x.x.x+x
```

```
export JAVA_HOME=/Library/Java/JavaVirtualMachines/temurin-  
21.jdk/Contents/Home (Mac OS X)
```

Instalación de Java en Linux

- Añadir el directorio *bin* de la distribución de Java a la variable de entorno **PATH**.
- Se recomienda añadirlo en la primera posición para evitar problemas con otras instalaciones de Java que puedan existir en el **PATH**.

```
export PATH=$JAVA_HOME/bin:$PATH
```

Instalación de Maven en Linux

- Descargar y descomprimir el fichero **TAR.GZ** de la siguiente dirección

`https://maven.apache.org/download.cgi`

- Se recomienda descomprimir la distribución en el directorio **/opt**.
- Establecer la variable de entorno **MAVEN_HOME**.

`MAVEN_HOME=/opt/apache-maven-3.x.x`

- Añadir el directorio **bin** de la distribución de Maven a la variable de entorno **PATH**.

`PATH=$MAVEN_HOME/bin:$PATH`

Compilación y ejecución de la aplicación web

- Descomprimir el fichero **ZIP** con la distribución de la aplicación web en un directorio con permisos de lectura y escritura.
- Para **compilar** los ejemplos, desde el directorio raíz de la aplicación ejecutar el siguiente comando:

```
$ mvn sql:execute install
```

- Para **ejecutar** la aplicación, desde ese mismo directorio raíz de la aplicación, ejecutar el siguiente comando:

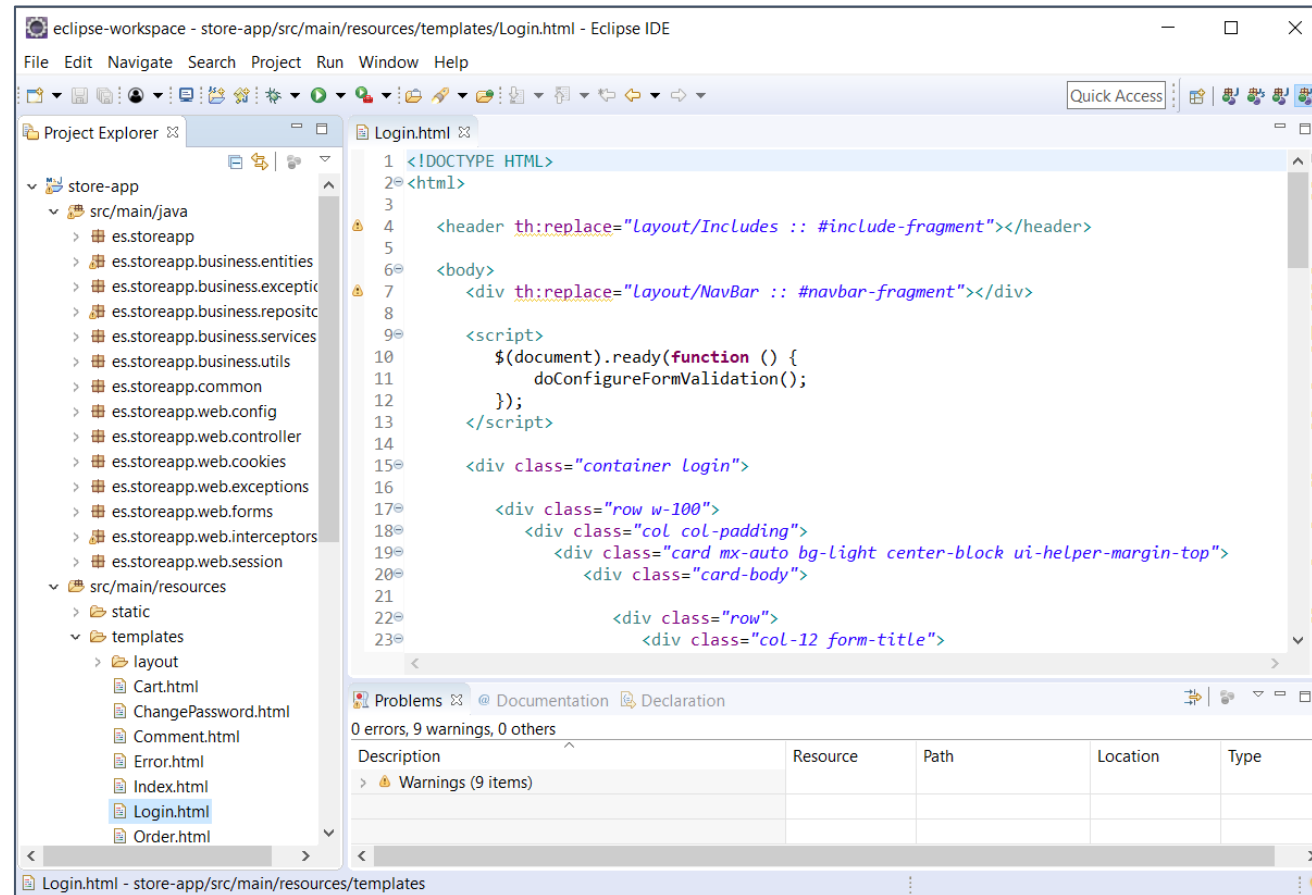
```
$ mvn spring-boot:run
```

- Acceder con el navegador a la dirección **`http://localhost:8888/`**

Eclipse (1)

- Acceder a <https://www.eclipse.org/downloads/packages/> y descargar la distribución **Eclipse IDE for Java EE Developers**.
- Descomprimir el fichero **ZIP** en un directorio con permisos de lectura y escritura, por ejemplo en la **\$HOME** del usuario.
- Abrir eclipse e importar el proyecto:
File -> Import -> Maven -> Existing Maven Projects
- Seleccionar la raíz del directorio que contiene la aplicación web.

Eclipse (2)

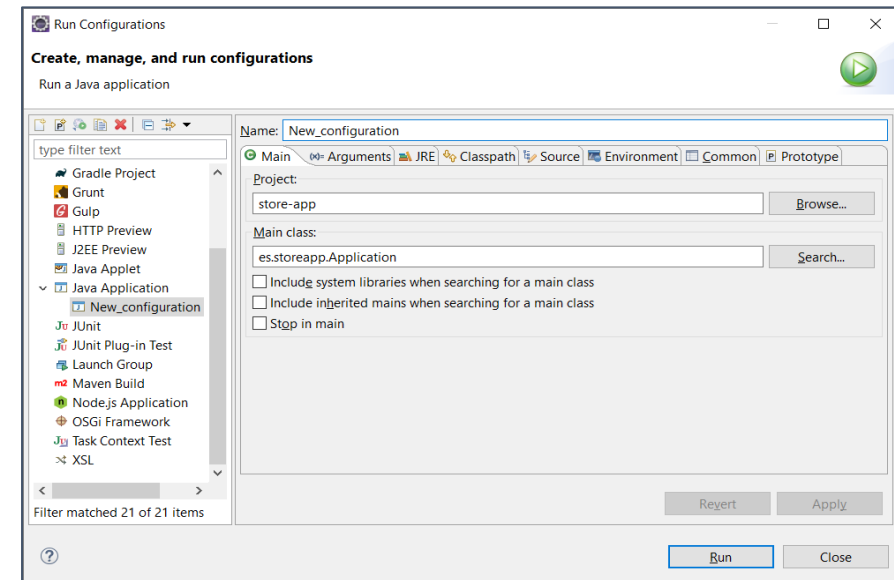


Eclipse (3)

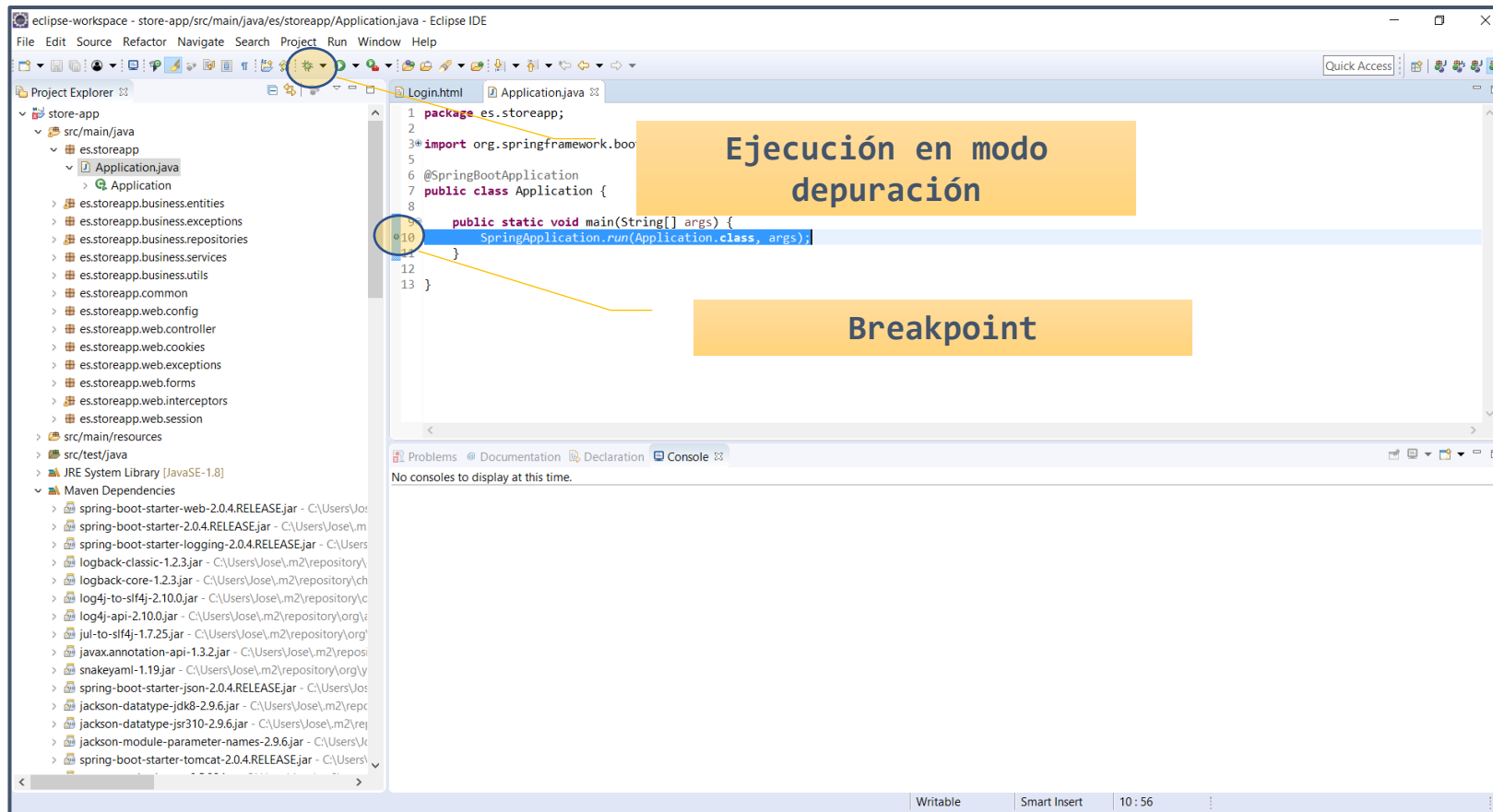
- Para ejecutar la aplicación se puede crear un lanzador en el menú de ejecución:

Run -> Run Configurations -> Java Application -> New Configuration (botón derecho)

- A continuación, es necesario seleccionar el proyecto **store-app** y la clase principal **es.storeapp.Application**.



Eclipse (4)

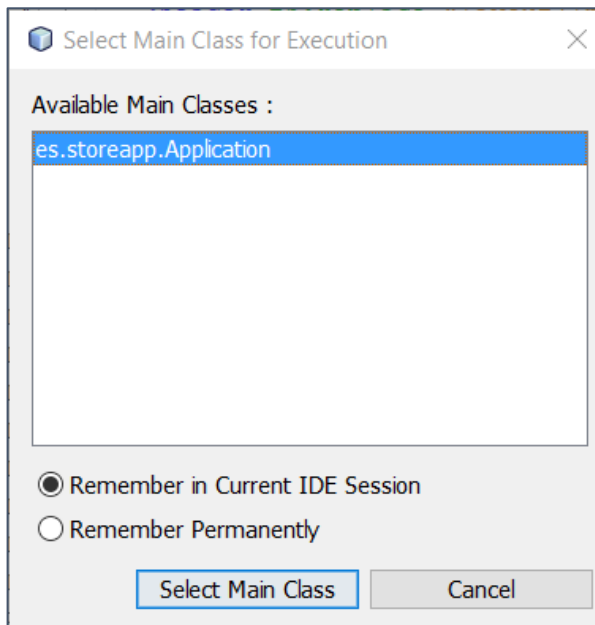


Netbeans (1)

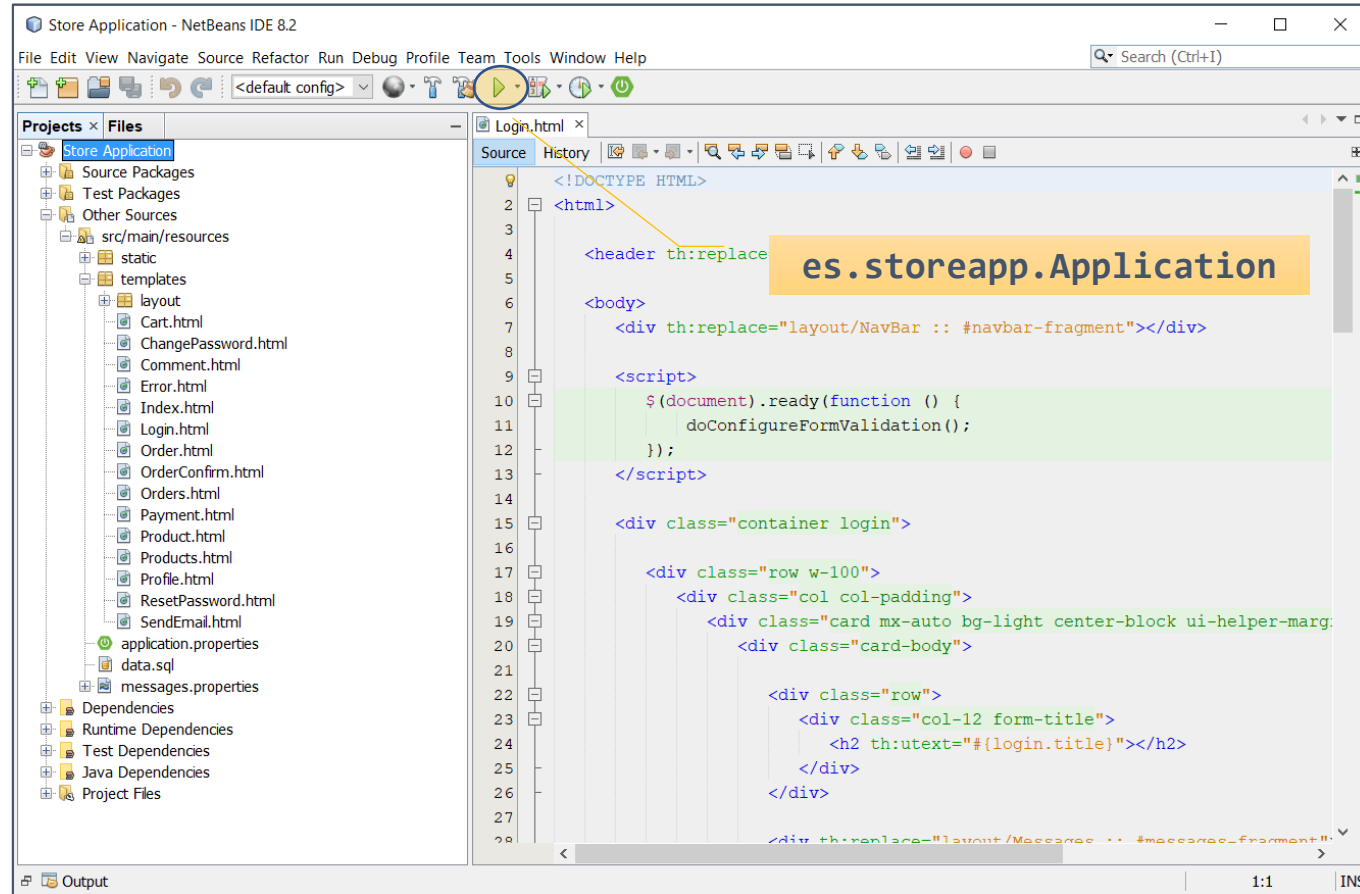
- Acceder a <https://netbeans.apache.org/download/index.html> (se recomienda utilizar la última versión).
- Instalar la aplicación en el directorio por defecto.
- Abrir Netbeans e importar el proyecto:
File -> Open Project
- Seleccionar la raíz del directorio que contiene la aplicación web.

Netbeans (2)

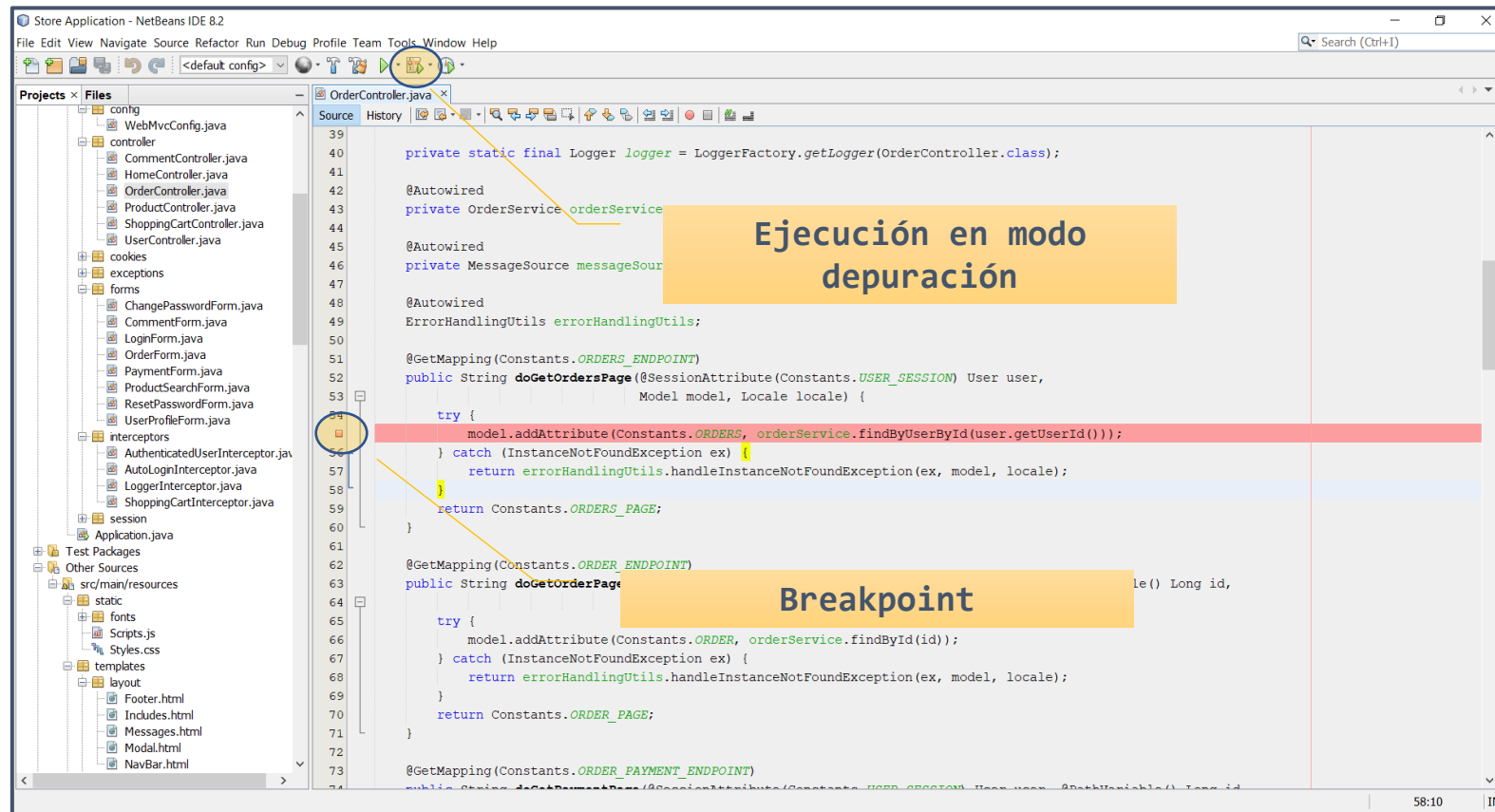
- Para ejecutar la aplicación, pulsar en el botón verde de ejecución y a continuación seleccionar la clase principal **es.storeapp.Application**. En versiones más recientes de Netbeans detectará la clase principal de forma automática.



Netbeans (3)



Netbeans (4)



Estructura del proyecto (1)

- La aplicación web se distribuye dentro de un proyecto Maven:
 - **pom.xml**: fichero de configuración del proyecto.
 - **src/main/java**: directorio de código fuente.
 - **src/main/resources**: ficheros de configuración.
 - **src/main/resources/templates**: plantillas HTML.
 - **src/main/resources/static**: recursos estáticos.
- El fichero principal de configuración de la aplicación es:
 - **src/main/resources/application.properties**

Estructura del proyecto (2): pom.xml

- Nombre del proyecto y versión.
- Librerías de código abierto que se usan en el proyecto (*dependencies*).
- Extensiones (*plugins*).

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apac
4      <modelVersion>4.0.0</modelVersion>
5
6      <groupId>es.storeapp</groupId>
7      <artifactId>store-app</artifactId>
8      <version>1.0.0</version>
9      <name>Store Application</name>
10
11     <parent>
16
17     <dependencies>
136
137     <properties>
138         <java.version>1.8</java.version>
139         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
140     </properties>
141
142     <build>
143         <plugins>
144             <plugin>
145                 <groupId>org.springframework.boot</groupId>
146                 <artifactId>spring-boot-maven-plugin</artifactId>
147             </plugin>
148             <plugin>
149                 <groupId>org.sonarsource.scanner.maven</groupId>
150                 <artifactId>sonar-maven-plugin</artifactId>
151                 <version>3.4.0.905</version>
152             </plugin>
153         </plugins>
154     </build>
155
156 </project>
```

Estructura del proyecto (3): application.properties

- En la siguiente figura se muestran los parámetros de configuración de la base de datos y de **JPA**.
- La aplicación utiliza la base de datos **Apache Derby** preconfigurada con todos los datos de productos y categorías pero sin usuarios registrados.
- El directorio de almacenamiento es **work/database**.

```
# =====  
# Datasource  
# =====  
  
spring.datasource.url=jdbc:derby:work/database;create=true  
spring.datasource.username=app  
spring.datasource.driver-class-name=org.apache.derby.jdbc.EmbeddedDriver  
spring.datasource.continue-on-error=false  
spring.datasource.initialization-mode=never  
  
# =====  
# JPA  
# =====  
  
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.format_sql=true  
spring.jpa.properties.hibernate.type=trace  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.DerbyTenSevenDialect  
spring.jpa.hibernate.ddl-auto=none
```

Estructura del proyecto (4): application.properties

- Para restaurar la base de datos a sus valores originales se puede seguir el siguiente procedimiento:
 - Parar la aplicación.
 - Desde una consola situada en la raíz del proyecto, ejecutar el siguiente comando:
\$ mvn sql:execute
 - Iniciar la aplicación.

Estructura del proyecto (5): application.properties

- La siguiente imagen muestra los parámetros de configuración del servidor, por ejemplo, el puerto, la página de error, etc.
- El directorio de trabajo es **work** en el que se almacenan, entre otros, las imágenes de perfil de los usuario registrados.

```
# =====  
# Custom properties  
# =====  
  
server.configuration.resources=work  
  
# =====  
# Web  
# =====  
  
server.port=8888  
server.error.path=/Error  
server.error.include-exception=true  
server.error.include-stacktrace=always
```

Estructura del proyecto (6): application.properties

- A continuación se muestran los parámetros de configuración para el envío de correos electrónicos, con un ejemplo para utilizar Gmail (el nombre de usuario y la contraseña aparecen ocultos).

```
server.configuration.mail-host=smtp.gmail.com
server.configuration.mail-port=587
server.configuration.mail-username=*****@gmail.com
server.configuration.mail-password=*****
server.configuration.mail-ssl-enable=true
server.configuration.mail-from=*****@gmail.com
```

Estructura del proyecto (7): application.properties

- Y en último lugar, se muestran los parámetros de configuración relacionados con el log y la monitorización de la aplicación.
- El fichero en el que se almacenarán los registros de log es **work/server.log**.

```
# =====  
# Logging  
# =====  
  
debug=false  
logging.level.es.storeapp=ERROR  
logging.level.org.springframework.web=ERROR  
logging.level.org.thymeleaf=ERROR  
logging.file=work/server.log  
  
# =====  
# Management  
# =====  
  
management.endpoints.web.exposure.include=*  
management.endpoint.shutdown.enabled=true
```

Estructura del proyecto (8): application.properties

- En el siguiente enlace ofrece información detalla sobre la configuración de Spring Boot:

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle>

- Los distintos parámetros de configuración se encuentran en un apéndice al final del documento:

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#common-application-properties>

Implementación (1): lógica de negocio

- La lógica de negocio de la aplicación está implementada utilizando Hibernate y JPA en el paquete **es.storeapp.business**.
- Las entidades persistentes de la aplicación se encuentran en el paquete **es.storeapp.business.entities**.
 - **User**: usuario registrado en la aplicación.
 - **Product**: producto que se puede adquirir en la tienda.
 - **Category**: categoría a la que pertenece un producto.
 - **Comment**: comentario realizado sobre un producto.
 - **Order**: pedido creado por un usuario registrado.
 - **OrderLine**: cada uno de los productos que se han comprado en el mismo pedido.
 - **OrderState**: estado actual del pedido.
 - **CreditCard**: tarjeta de crédito.

Implementación (2): lógica de negocio

- Los casos de uso de la aplicación se encuentran implementados en los servicios localizados en el paquete **es.storeapp.business.services**.
 - **UserService**: casos de uso relacionados con las funcionalidades del usuario. Por ejemplo, creación de usuarios, autenticación, cambio de contraseña, etc.
 - **ProductService**: casos de uso relacionados con productos y sus comentarios.
 - **OrderService**: casos de uso relacionados con la gestión de los pedidos. Por ejemplo, creación de pedidos, pagos, cancelaciones, etc.
- Las excepciones que pueden lanzar los servicios se encuentran en el paquete **es.storeapp.business.exceptions**.

Implementación (3): lógica de negocio

- Las repositorios ofrecen un API de bajo nivel para trabajar con las principales entidades persistentes de la aplicación (creación, actualización, borrado, etc.) y se encuentran en el paquete **es.storeapp.business.repositories**.
 - **UserRepository**: repositorio de usuarios.
 - **ProductRepository**: repositorio de productos.
 - **CategoryRepository**: repositorio de categorías.
 - **CommentRepository**: repositorio de comentarios.
 - **OrderRepository**: repositorio de pedidos.
 - **OrderLineRepository**: repositorio de líneas de pedido.
- Todos ellos heredan de un repositorio raíz **AbstractRepository**.

Implementación (4): lógica de negocio

- Para construir las consultas que se ejecutan contra la base de datos se utiliza **JPQL (Java Persistence Query Language)**, un lenguaje de consulta que forma parte de la especificación de **JPA (Java Persistence API)**.
- **JPQL** se puede considerar como la versión orientada a objetos de SQL, por lo tanto, los desarrolladores familiarizados con SQL, no tendrán mayores dificultades es trabajar con JPQL.

```
1 Query query = em.createQuery("SELECT e FROM Employee e WHERE e.salary > 100000");  
2 List<Employee> result = query.getResultList();
```

Implementación (5): subsistema controlador

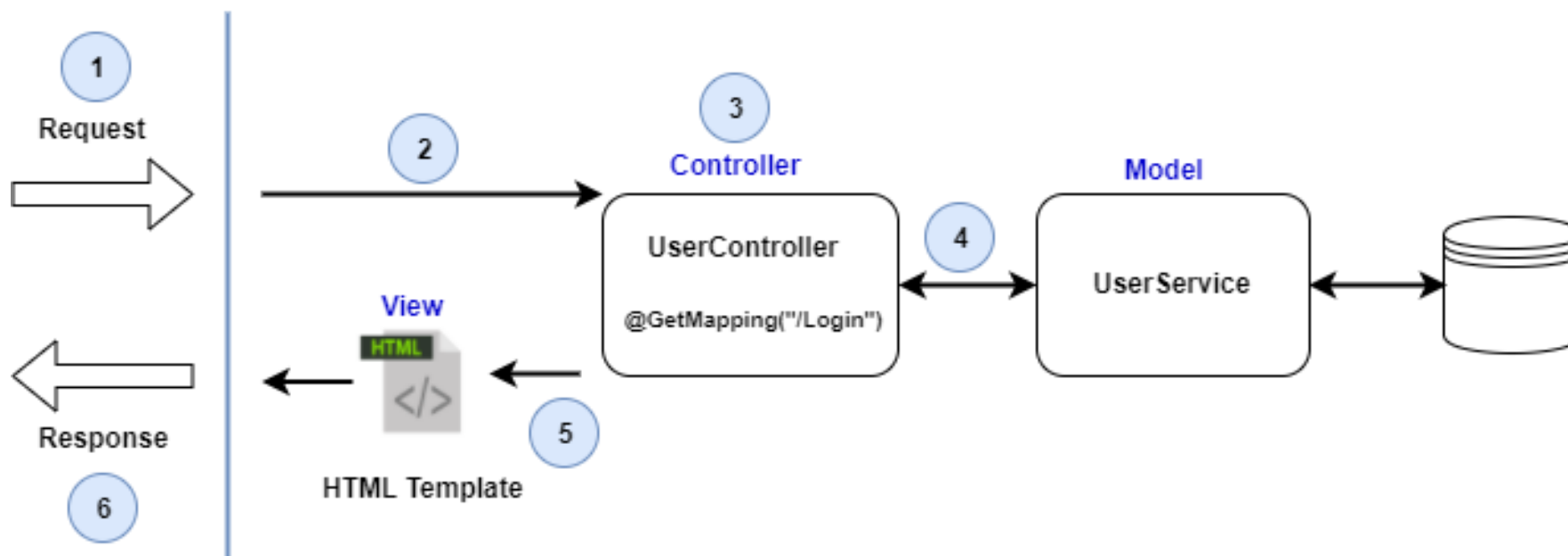
- El subsistema controlador es el encargado de gestionar las peticiones HTTP que llegan desde los clientes web, realizar las llamadas a la lógica de negocio y finalmente devolver los resultados en formato HTML.
- Este subsistema está implementado utilizando Spring MVC y se encuentra en el paquete **es.storeapp.web**.
 - **config**: configuración básica de seguridad e interceptores.
 - **controllers**: clases encargadas de procesar las peticiones HTTP (*endpoints*).
 - **cookies**: clase que se utiliza para guardar la cookie persistente.
 - **forms**: clases que reciben los datos que proceden de los formularios.
 - **interceptors**: clases que se ejecutan antes y después de los controladores.
 - **session**: clase que guarda el carrito de la compra en memoria.

Implementación (6): subsistema controlador

- Los controladores son las clases encargadas de procesar las peticiones HTTP, realizar las llamadas a la lógica de negocio y finalmente, transferir el control a la plantilla HTML en la que se mostrarán los resultados:
 - **HomeController**: página principal de la aplicación.
 - **UserController**: peticiones relacionadas con las funciones del usuario.
 - **ProductController**: peticiones relacionadas con los productos.
 - **CommentController**: peticiones relacionadas con los comentarios.
 - **ShoppingCartController**: peticiones relacionadas con el carrito.
 - **OrderController**: peticiones relacionadas con las compras.

Implementación (7): subsistema controlador

- El siguiente diagrama muestra el flujo de interacciones entre subsistemas desde que el navegador envía la petición hasta que se genera la respuesta HTML.



Implementación (8): subsistema controlador

1. La petición se envía desde el cliente web.
2. Una vez que llega al servidor, en primer lugar se ejecutan los interceptores.
3. A continuación la petición llega al controlador.
4. El controlador accede a la lógica de negocio para obtener los datos necesarios que le permitan resolver la petición.
5. Se transfiere el control al motor de plantillas quien se encarga de la generación del contenido HTML.
6. Se envía la respuesta al cliente web.

Implementación (9): subsistema controlador

Formulario de entrada

```
@PostMapping(Constants.REGISTRATION_ENDPOINT)
public String doRegister(@Valid @ModelAttribute(Constants.USER_PROFILE_FORM) UserProfileForm userProfileForm,
    BindingResult result,
    RedirectAttributes redirectAttributes,
    HttpSession session,
    Locale locale, Model model) {

    if (result.hasErrors()) {
        errorHandlingUtils.handleInvalidFormError(result,
            Constants.REGISTRATION_INVALID_PARAMS_MESSAGE, model, locale);
        return Constants.USER_PROFILE_PAGE;
    }

    User user;
    try {
        user = userService.create(userProfileForm.getName(), userProfileForm.getEmail(),
            userProfileForm.getPassword(), userProfileForm.getAddress(),
            userProfileForm.getImage() != null ? userProfileForm.getImage().getOriginalFilename() : null,
            userProfileForm.getImage() != null ? userProfileForm.getImage().getBytes() : null);
        session.setAttribute(Constants.USER_SESSION, user);
        redirectAttributes.addFlashAttribute(Constants.SUCCESS_MESSAGE, messageSource.getMessage(
            Constants.REGISTRATION_SUCCESS_MESSAGE, new Object[]{user.getName()}, locale));
    } catch (DuplicatedResourceException ex) {
        return errorHandlingUtils.handleDuplicatedResourceException(ex, Constants.USER_PROFILE_PAGE,
            model, locale);
    } catch (IOException ex) {
        return errorHandlingUtils.handleUnexpectedException(ex, model);
    }

    return Constants.SEND_REDIRECT + Constants.ROOT_ENDPOINT;
}
```

Sesión

Validación del formulario

Llamada al modelo

Plantilla de la página principal

Implementación (10): subsistema controlador

- Los distintos formularios se encuentran en el paquete **es.storeapp.web.forms**.
- Estas clases Java se generan de forma automática a partir de las peticiones HTTP.
 - **UserProfileForm**: registrar un nuevo usuario o actualizar su perfil.
 - **LoginForm**: autenticación.
 - **ChangePasswordForm**: cambiar la contraseña.
 - **ResetPasswordForm**: resetear la contraseña.
 - **ProductSearchForm**: búsqueda de productos por categoría.
 - **OrderForm**: comprar un pedido.
 - **PaymentForm**: pagar un pedido.
 - **CommentForm**: comentar un producto.

Implementación (11): subsistema controlador

- Los atributos de las clases que representan formularios, se rellenan de forma automática a partir de los nombres de las etiquetas *input* del código HTML.

```
8 public class UserProfileForm {
9
10     @NotNull
11     @Size(min=4)
12     private String name;
13
14     @NotNull
15     @Email
16     private String email;
17
18     private String password;
19
20     @NotNull
21     private String address;
22
23     private MultipartFile image;
24
25     public UserProfileForm() {
26     }
```

```
<div class="form-group row">
  <label for="email" class="col-3 text-right input-label-middle">Email a
  <div class="input-group col-9 col-sm-6">
    <div class="input-group-prepend">
      <span class="input-group-text"><i class="fas fa-at"></i></span>
    </div>
    <input type="email" class="form-control" id="email" name="email"
      aria-describedby="emailHelp"
      placeholder="Enter email"
      data-validation-error-msg-container="#input-email-error"
      data-validation="email" data-validation-error-msg="The email
    </div>
    <div class="d-none d-sm-block col-sm-3">
      <small id="emailHelp" class="form-text text-muted text-left">We'll
    </div>
  </div>
  <div class="form-group row d-none">
    <div class="col-3"></div>
    <div class="col-9 text-left" id="input-email-error"></div>
  </div>
```

Implementación (12): subsistema controlador

- Las validaciones más sencillas de los campos de los formularios se pueden realizar con las anotaciones predefinidas en el API de Java **javax.validation**.

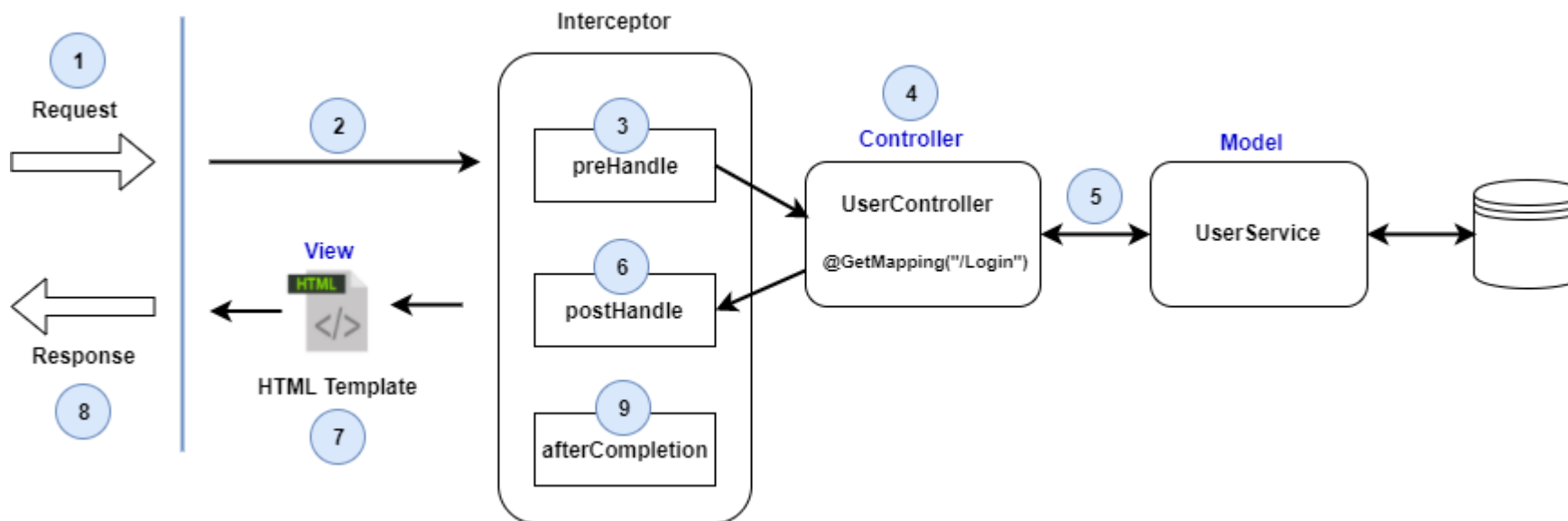
<https://www.baeldung.com/javax-validation>

- También es posible implementar validaciones a medida:

<https://www.baeldung.com/spring-mvc-custom-validator>

Implementación (13): subsistema controlador

- El siguiente diagrama muestra el funcionamiento de los interceptores y sus llamadas **preHandle** y **postHandle** que se ejecutan antes y después de la llamada al controlador.



Implementación (14): subsistema controlador

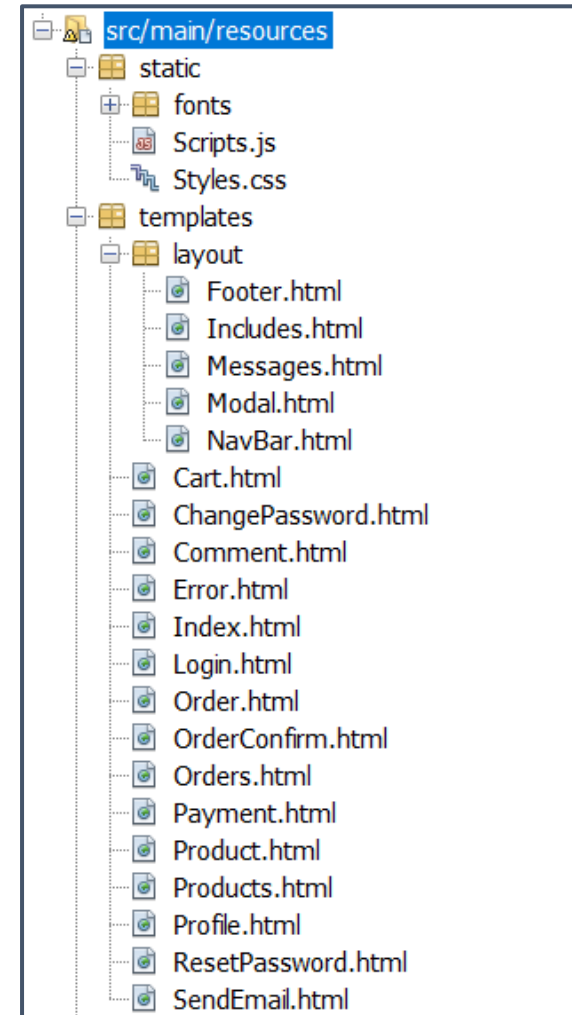
- Los distintos interceptores definidos en la aplicación web se encuentran en el paquete **es.storeapp.web.interceptors**.
 - **AutoLoginInterceptor**: detecta si está presente la cookie de sesión y en caso afirmativo realiza el proceso de autenticación de forma automática.
 - **ShoppingCartInterceptor**: garantiza que el carrito de la compra siempre está presente en la sesión.
 - **LoggerInterceptor**: escribe un registro de log por cada petición HTTP.
 - **AuthenticatedUserInterceptor**: este interceptor comprueba que existe un usuario en la sesión antes de permitir el acceso a los recursos que requieren autenticación.
 - El método *preHandle* devuelve *false* si un usuario no autenticado intenta acceder a un recurso protegido.
 - La lista de recursos protegidos por este interceptor se define en la clase **es.storeapp.web.config.WebMvcConfig**: perfil de usuario, cambio de contraseña, pedidos y comentarios de productos.

Implementación (15): subsistema vista

- El motor de plantillas utilizado en esta aplicación es Thymeleaf:
<https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html>
- Las diferentes plantillas se encuentran en el directorio **src/main/resources/templates**.
- Los fragmentos comunes a todas las plantillas se encuentran en el directorio **src/main/resources/templates/layout**.
 - **Navbar.html**: barra de navegación de la parte superior.
 - **Includes.html**: inclusión de ficheros de recursos JavaScript, CSS, etc.
 - **Footer.html**: pie de página.
 - **Messages.html**: plantillas para distintos tipos de mensajes.
 - **Modal.html**: plantilla para diálogos modales.

Implementación (16): subsistema vista

- Las siguiente figura muestra todas las plantillas que se utilizan en la aplicación.
- En el directorio **static** se encuentran recursos estáticos de acceso público (por ejemplo, fuentes).



Implementación (17): subsistema vista

- El siguiente ejemplo ilustra el funcionamiento básico de Thymeleaf en la página que muestra el listado de productos por categoría.
- En primer lugar, en el controlador Java se accede a base de datos para realizar la búsqueda de productos por categoría.
- A continuación se añaden los productos al contexto de ejecución (la clase **Model**).

```
@GetMapping(value = {Constants.PRODUCTS_ENDPOINT})  
public String doGetProductsPage(@ModelAttribute ProductSearchForm searchForm, Model model) {  
    List<Product> products = productService.findProducts(searchForm.getCategory());  
    List<Category> categories = productService.findAllCategories();  
    model.addAttribute(Constants.PRODUCTS, products);  
    model.addAttribute(Constants.CATEGORIES, categories);  
    return Constants.PRODUCTS_PAGE;  
}
```

Implementación (18): subsistema vista

- El contexto de ejecución es un **mapa** al que se ha añadido la lista de productos utilizando la clave “*products*”.
- La etiqueta **th:each** busca en el contexto una lista identificada por la clave “*products*”.

```
<tbody>
  <tr th:each="product,status" #{products}>
    <td class="align-middle"><i class="fas" th:classappend="${product.icon}"></i></td>
    <td class="align-middle">
      <span th:utext="${product.name}" th:title="${product.description}"></span>
    </td>
    <td th:utext="${product.description}"></td>
    <td class="align-middle" th:utext="${product.category.name}"></td>
    <td class="align-middle" th:attr="data-order=${product.rating}">
      <input type="text" class="rating-loading" th:value="${product.rating}" />
    </td>
    <td class="align-middle"><span th:utext="${product.sales}"></span></td>
    <td class="align-middle"><span th:utext="${product.price}"></span><span>&euro;</span></td>
    <td class="align-middle">
      <form method="GET" th:action="@{/products/{id} (id=${product.productId})}">
        <button class="btn btn-sm btn-success" type="submit">
          <i class="fa fa-search"></i>
        </button>
      </form>
    </td>
  </tr>
</tbody>
```

Implementación (19): subsistema vista

- Y a continuación, la plantilla itera por la lista de productos para generar el código HTML de cada uno de ellos.

```
<td class="align-middle">
  <span th:utext="${product.name}" th:title="${product.description}"></span>
</td>
<td th:utext="${product.description}"></td>
<td class="align-middle" th:utext="${product.category.name}"></td>
<td class="align-middle" th:attr="data-order=${product.rating}">
  <input type="text" class="rating-loading" th:value="${product.rating}" />
</td>
<td class="align-middle"><span th:utext="${product.sales}"></span></td>
<td class="align-middle"><span th:utext="${product.price}"></span><span>&euro;</span></td>
```

Implementación (20): subsistema vista

- Para acceder a los datos de un producto, se utilizan directamente los nombres de los atributos (las entidades tienen **getters** y **setters** para todos sus atributos).
- Por simplicidad del código, la capa vista accede directamente a las entidades del modelo. En una aplicación real deberían de exponerse DTO (**Data Transfer Objects**) y no directamente las entidades del modelo.

```
@Entity(name = Constants.PRODUCT_ENTITY)
@Table(name = Constants.PRODUCTS_TABLE)
public class Product implements Serializable{

    private static final long serialVersionUID = 7007987631L;

    @Id
    private Long productId;

    @ManyToOne
    @JoinColumn(name = "category", nullable = false)
    private Category category;

    @Column(name = "name", nullable = false, unique = true)
    private String name;

    @Column(name = "description", nullable = false)
    private String description;

    @Column(name = "icon", nullable = false)
    private String icon;

    @Column(name = "price", nullable = false)
    private Integer price;

    @Column(name = "totalScore", nullable = false)
    private Integer totalScore;

    @Column(name = "totalComments", nullable = false)
    private Integer totalComments;

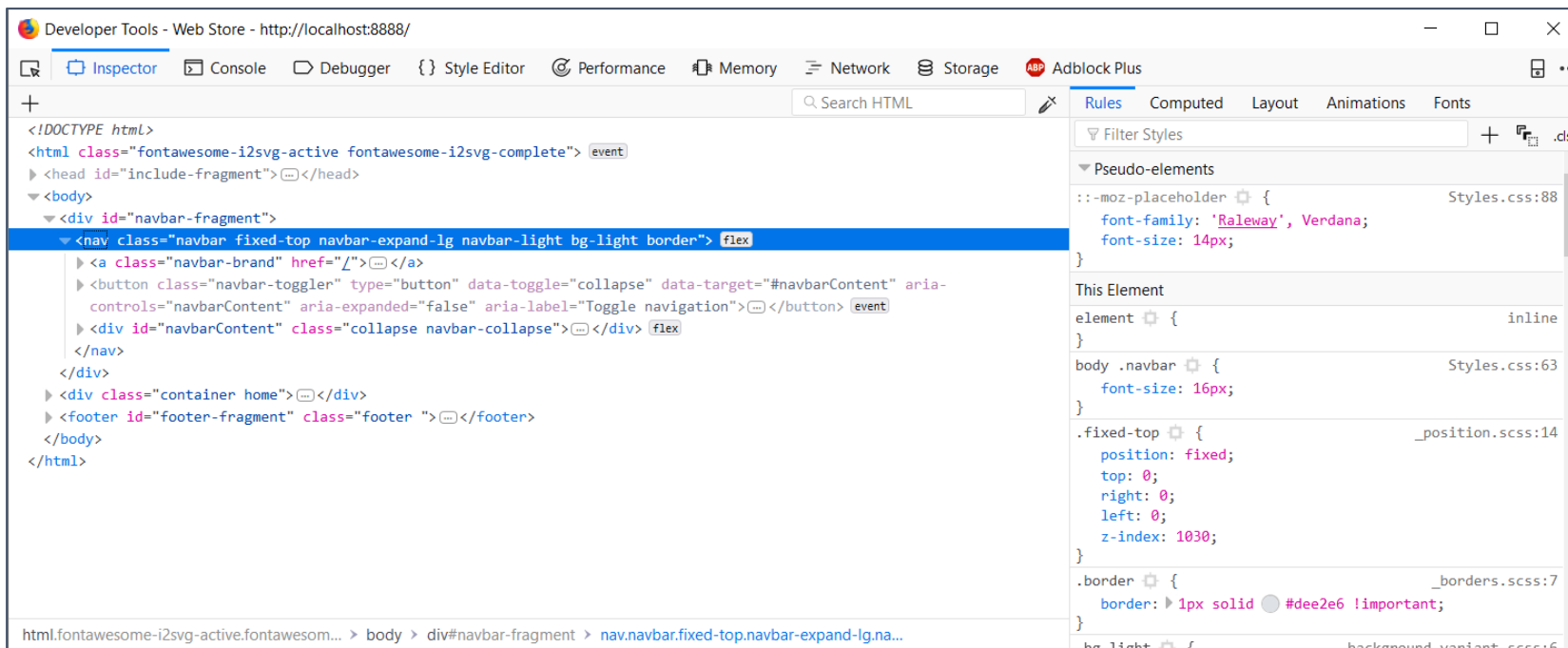
    @Column(name = "sales", nullable = false)
    private Integer sales;
```

Herramientas de desarrollador del navegador (1)

- Todos los navegadores incluyen herramientas de desarrollador que pueden ser muy útiles a la hora de realizar la práctica.
- Para abrir estas herramientas se puede hacer a través de la tecla **F12**.
- A continuación se abre una ventana que incluye diferentes pestañas con diferentes utilidades:
 - Inspeccionar el árbol DOM.
 - Consola de ejecución de JavaScript.
 - Depurador de JavaScript.
 - Log de peticiones HTTP.

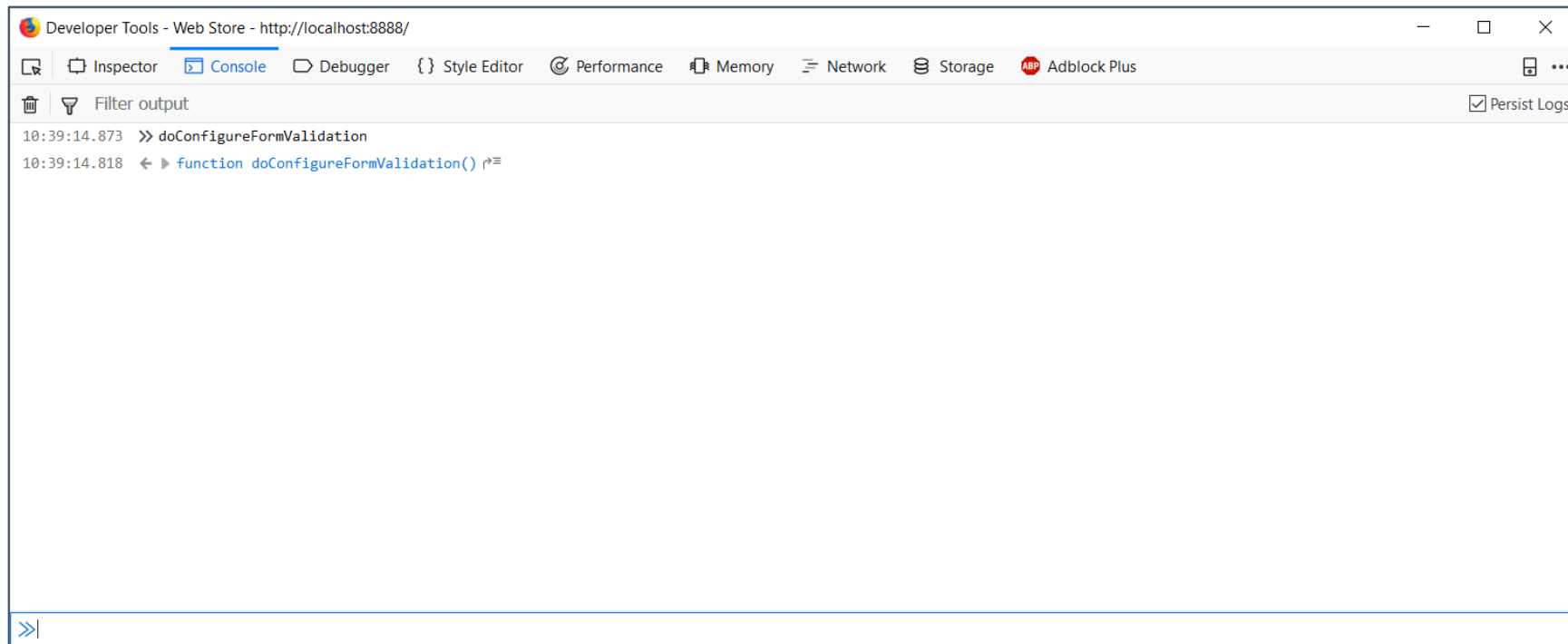
Herramientas de desarrollador del navegador (2)

- El **inspector** del árbol DOM permite ver la estructura de los nodos que se han cargado en el documento. Podría ser útil, por ejemplo, para detectar si hay algún script inyectado.



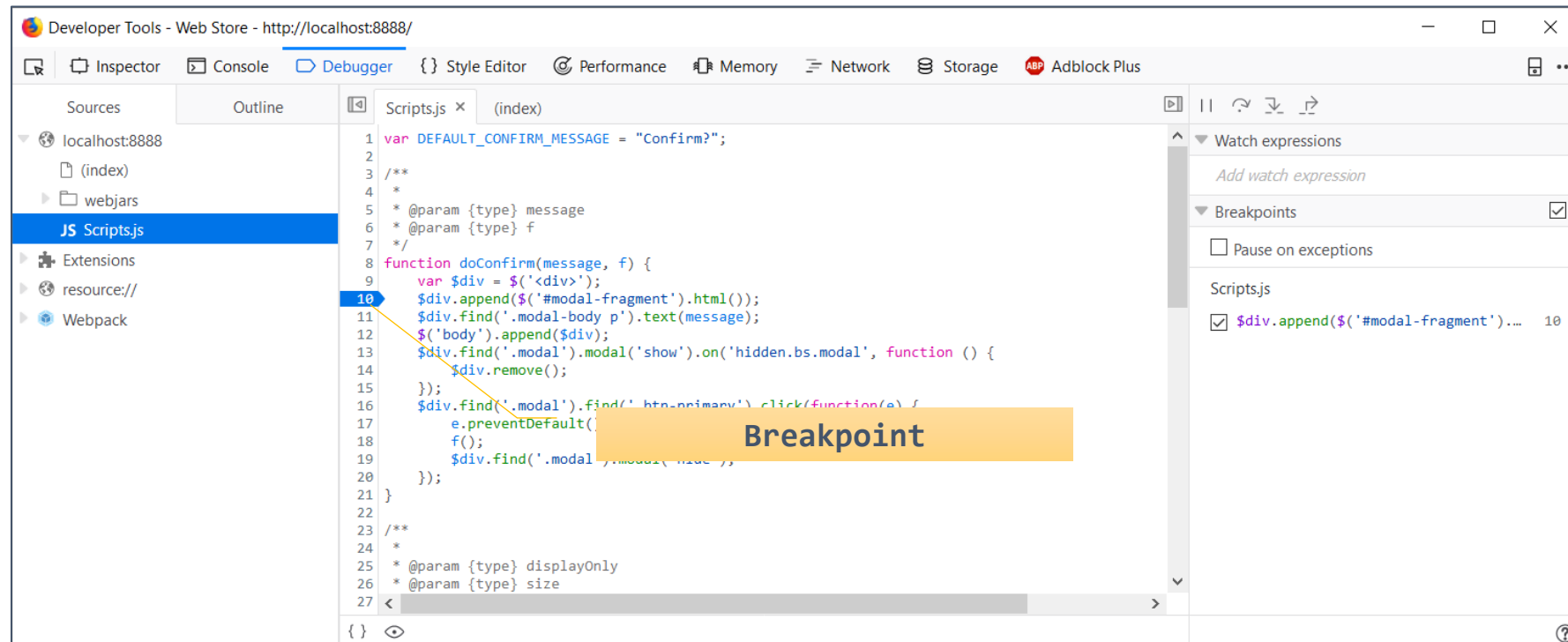
Herramientas de desarrollador del navegador (3)

- La **consola** permite ejecutar cualquier código JavaScript así como ver las variables y funciones definidas en el contexto de la aplicación.



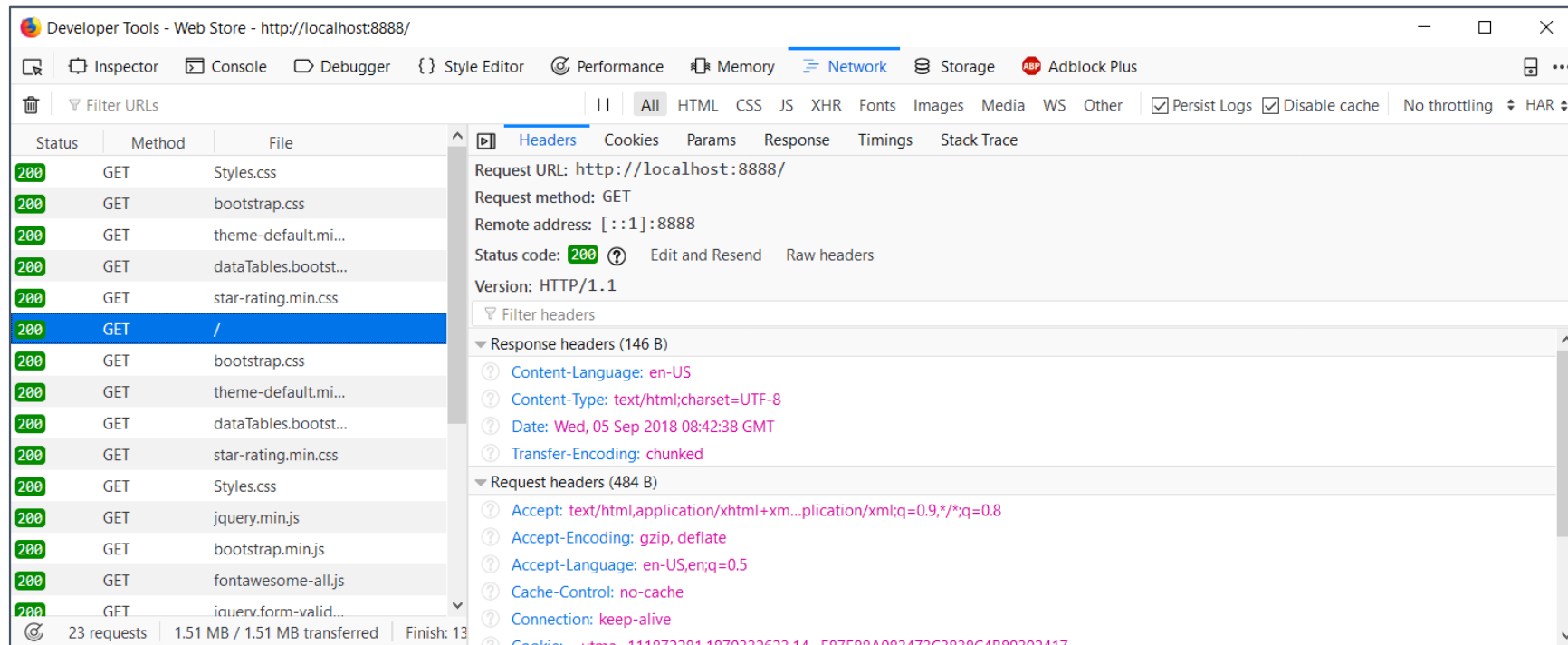
Herramientas de desarrollador del navegador (4)

- El **depurador** permite establecer *breakpoints* en el código JavaScript para ver el estado de la ejecución en un determinado momento.



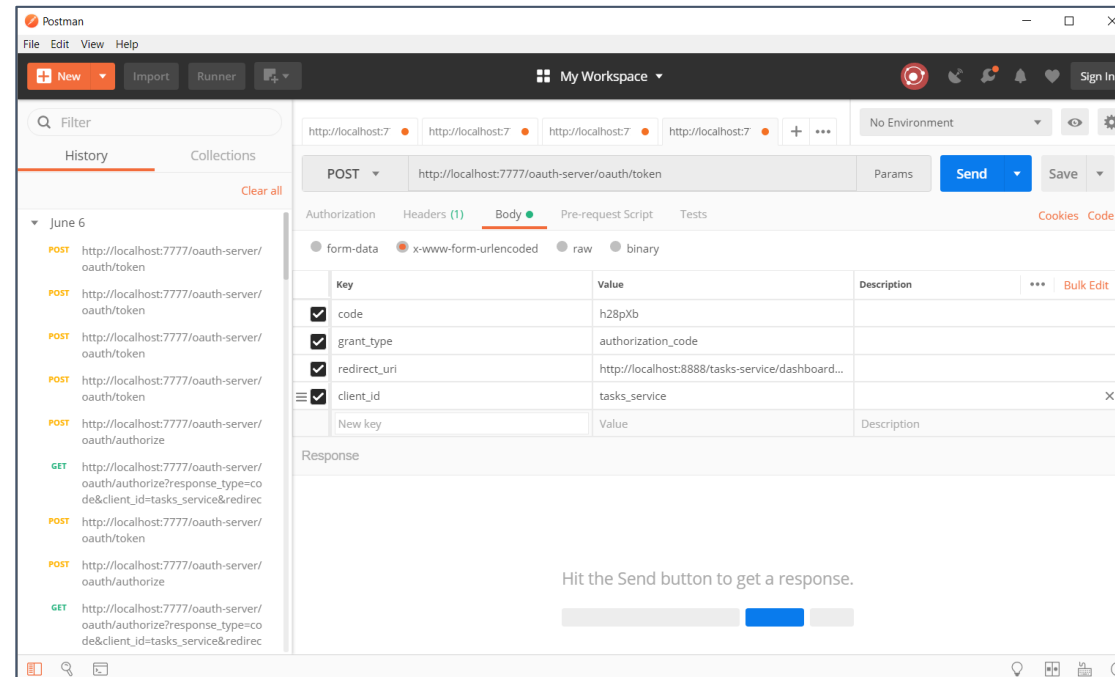
Herramientas de desarrollador del navegador (5)

- El log de **red** permite monitorizar todas las peticiones HTTP que se ejecutan desde el navegador.



Postman (1)

- Para detectar y probar algunas vulnerabilidades puede ser necesario enviar peticiones HTTP directamente sin utilizar el navegador web.
- Una de las herramientas más utilizadas para enviar peticiones HTTP es **Postman**.



Postman (2)

- Postman permite construir todo tipo de peticiones HTTP (no sólo **GET** y **POST**).
- También permite modificar cualquier cabecera HTTP (por ejemplo, **Content-Type**, **Cookie**, **Authentication**) y para algunas de las ellas ofrece un interfaz de alto nivel.
- Es posible enviar todo tipo de peticiones POST desde un interfaz de usuario amigable, desde formularios (**x-www-form-urlencoded** o **form-data**) hasta cualquier otro tipo de contenido .

Wireshark y RawCap

- **Wireshark** es una herramienta que permite capturar el tráfico que circula por un segmento de red.

<https://www.wireshark.org/download.html>

- Para capturar el tráfico local en sistemas Windows, se puede utilizar la herramienta **RawCap**. Los ficheros que genera se pueden importar en Wireshark.

<https://www.netresec.com/?page=RawCap>